

Design and Implementation of IoT-Enabled Self-Balancing Robot Using ESP32 with FreeRTOS Multitasking

Le Nhat Huy*, Ngo Gia Bao, Tran Hao Khiem, Nguyen Le Hai Long,
Nguyen The Bao, Truong Quang Phuc

¹Ho Chi Minh City University of Technology and Engineering, Vietnam

*Corresponding author. Email: 23119064@student.hcmute.edu.vn

ARTICLE INFO

Received: 31/12/2026
Revised: 10/01/2026
Accepted: 14/01/2026
Published: 15/01/2026

KEYWORDS

Self-balancing robot;
ESP32;
FreeRTOS;
MQTT;
LQR control.

ABSTRACT

This paper presents the design and implementation of an IoT-enabled self-balancing two-wheeled robot using ESP32 microcontroller with FreeRTOS real-time operating system. The system integrates multiple communication channels including MQTT over TLS for secure cloud connectivity, Firebase Realtime Database for data logging, and Bluetooth for local control. A Linear Quadratic Regulator (LQR) control algorithm combined with Kalman filtering for sensor fusion achieves stable balance control. The FreeRTOS multitasking architecture distributes workload across ESP32 dual cores: the balance control task runs at 3ms period on Core 1 for hard real-time requirements, while communication tasks (MQTT, Firebase) operate on Core 0. Experimental results demonstrate stable balancing within $\pm 5^\circ$ deviation and successful cloud telemetry transmission. The proposed architecture provides a scalable framework for IoT-enabled mobile robots with deterministic control performance.

Thiết kế và triển khai robot tự cân bằng tích hợp IoT sử dụng ESP32 với đa nhiệm FreeRTOS

Lê Nhật Huy^{1*}, Ngô Gia Bảo, Trần Hạo Khiêm, Nguyễn Lê Hải Long,
Nguyễn Thế Bảo, Trương Quang Phúc

¹Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, Việt Nam

*Tác giả liên hệ. Email: 23119064@student.hcmute.edu.vn

THÔNG TIN BÀI BÁO

Ngày nhận bài: 31/12/2026
Ngày hoàn thiện: 10/01/2026
Ngày chấp nhận: 14/01/2026
Ngày đăng: 15/01/2026

TỪ KHÓA

Robot tự cân bằng;
ESP32;
FreeRTOS;
MQTT;
Điều khiển LQR.

TÓM TẮT

Bài báo này trình bày thiết kế và triển khai robot tự cân bằng hai bánh tích hợp IoT sử dụng vi điều khiển ESP32 với hệ điều hành thời gian thực FreeRTOS. Hệ thống tích hợp nhiều kênh truyền thông bao gồm MQTT qua TLS để kết nối đám mây bảo mật, Firebase Realtime Database để ghi dữ liệu, và Bluetooth để điều khiển cục bộ. Thuật toán điều khiển Linear Quadratic Regulator (LQR) kết hợp với bộ lọc Kalman để tổng hợp cảm biến đạt được điều khiển cân bằng ổn định. Kiến trúc đa nhiệm FreeRTOS phân phối khối lượng công việc trên hai lõi ESP32: tác vụ điều khiển cân bằng chạy với chu kỳ 3ms trên Core 1 đáp ứng yêu cầu thời gian thực cứng, trong khi các tác vụ truyền thông (MQTT, Firebase) hoạt động trên Core 0. Kết quả thực nghiệm cho thấy cân bằng ổn định trong phạm vi sai lệch $\pm 5^\circ$ và truyền dữ liệu đám mây thành công. Kiến trúc đề xuất cung cấp khung mở rộng cho robot di động tích hợp IoT với hiệu năng điều khiển xác định.

1. Giới thiệu

Robot tự cân bằng hai bánh là một bài toán điều khiển kinh điển trong lĩnh vực robotics, đòi hỏi sự kết hợp giữa cảm biến, thuật toán điều khiển và hệ thống thời gian thực [1]. Với sự phát triển của Internet of Things (IoT), việc tích hợp khả năng kết nối đám mây vào robot di động mở ra nhiều ứng dụng mới trong giám sát từ xa, thu thập dữ liệu và điều khiển phân tán [2].

ESP32 là vi điều khiển hai lõi với tích hợp WiFi và Bluetooth, phù hợp cho các ứng dụng IoT đòi hỏi xử lý thời gian thực [3]. FreeRTOS cung cấp khả năng đa nhiệm với lập lịch ưu tiên, cho phép phân chia rõ ràng giữa tác vụ điều khiển thời gian thực và truyền thông [4].

Bài báo này trình bày thiết kế một hệ thống robot tự cân bằng tích hợp đầy đủ các tính năng: điều khiển LQR với bộ lọc Kalman [5], truyền thông MQTT qua TLS đến HiveMQ Cloud [6], lưu trữ dữ liệu Firebase [7], và điều khiển Bluetooth. Kiến trúc FreeRTOS được thiết kế để đảm bảo tính xác định của vòng điều khiển trong khi vẫn duy trì kết nối IoT ổn định.

2. Phương pháp nghiên cứu

Phần này mô tả chi tiết phần cứng, thuật toán điều khiển và kiến trúc phần mềm của hệ thống.

2.1. Hệ thống phần cứng

Hệ thống phần cứng bao gồm vi điều khiển ESP32 WROOM-32, cảm biến IMU MPU6050 (gia tốc kế 3 trục và con quay hồi chuyển 3 trục), module điều khiển động cơ L298N, hai động cơ DC với encoder quang học, và nguồn pin LiPo 11,1V. Sơ đồ kết nối được mô tả trong Bảng 1.

Bảng 1. Kết nối phần cứng hệ thống

Thành phần	Chân GPIO	Ghi chú
MPU6050 (I2C)	SDA: 21, SCL: 22	400kHz I2C clock
Motor L (L298N)	ENA: 25, IN1: 27, IN2: 26	PWM điều khiển tốc độ
Motor R (L298N)	ENB: 14, IN3: 12, IN4: 13	PWM điều khiển tốc độ
Encoder L	A: 34, B: 35	Interrupt RISING
Encoder R	A: 32, B: 33	Interrupt RISING

2.2. Thuật toán điều khiển

Thuật toán điều khiển sử dụng bộ điều khiển Linear Quadratic Regulator (LQR) đơn giản hóa với các biến trạng thái: góc nghiêng thân xe (ψ), vận tốc góc nghiêng ($\dot{\psi}$), vị trí bánh xe (θ), và vận tốc bánh xe ($\dot{\theta}$). Tín hiệu điều khiển được tính theo công thức:

$$u = K_1\theta + K_2\dot{\theta} + K_3\psi + K_4\dot{\psi}$$

Với các hệ số $K_3 = 800$ và $K_4 = 1,5$ được tinh chỉnh thực nghiệm. Góc nghiêng được ước lượng từ dữ liệu MPU6050 sử dụng bộ lọc Kalman với các tham số: $Q_{angle} = 0,0000085$, $Q_{bias} = 0,000005$, $R_{measure} = 0,0009$.

2.3. Kiến trúc phần mềm FreeRTOS

Kiến trúc phần mềm sử dụng FreeRTOS với bốn tác vụ chính được phân bổ trên hai lõi CPU như mô tả trong Bảng 2. Tác vụ Balance chạy trên Core 1 với độ ưu tiên cao nhất, đảm bảo tính thời gian thực cho vòng điều khiển. Các tác vụ truyền thông chạy trên Core 0 để không ảnh hưởng đến điều khiển.

Bảng 2. Cấu hình các tác vụ FreeRTOS

Tác vụ	Core	Priority	Chu kỳ	Stack
TaskBalance	1	5	3 ms	4096 B
TaskControl	0	2	20 ms	2048 B
TaskMQTT	0	2	50 ms	8192 B
TaskFirebase	0	1	100 ms	8192 B

Mutex được sử dụng để bảo vệ các biến chia sẻ giữa các tác vụ, tránh race condition. Hàm `vTaskDelayUntil()` đảm bảo chu kỳ chính xác cho các tác vụ.

2.4. Tích hợp IoT

MQTT:

Giao thức MQTT (Message Queuing Telemetry Transport) được lựa chọn cho truyền thông thời gian thực nhờ đặc tính lightweight và mô hình publish/subscribe phù hợp với thiết bị IoT tài nguyên hạn chế. Hệ thống kết nối đến HiveMQ Cloud thông qua TLS trên port 8883, đảm bảo mã hóa end-to-end cho dữ liệu truyền tải. Cấu hình kết nối sử dụng `WiFiClientSecure` với chế độ `setInsecure()` để bỏ qua xác thực certificate, phù hợp cho môi trường phát triển.

Cấu trúc topic được thiết kế theo pattern `balancebot1/{device_id}/{channel}` với bốn kênh chính: `telemetry` để publish dữ liệu cảm biến định kỳ, `command` để nhận lệnh điều khiển từ cloud, `status` để thông báo trạng thái online/offline với Last Will Testament, và `config` để cập nhật tham số điều khiển từ xa. Last Will Testament được cấu hình với message `{"online":false}` và retain flag, đảm bảo hệ thống giám sát biết được khi robot mất kết nối đột ngột.

Telemetry data được đóng gói dưới dạng JSON với các trường: `ang` (góc nghiêng), `pwmL` và `pwmR` (tín hiệu PWM), `encL` và `encR` (giá trị encoder), `fall` (trạng thái ngã), và `bal` (trạng thái cân bằng). Việc sử dụng tên trường ngắn gọn giúp giảm kích thước payload, quan trọng cho băng thông hạn chế. Buffer size được cấu hình 512 bytes và keep-alive 60 giây để duy trì kết nối ổn định.

Xử lý command từ MQTT hỗ trợ các lệnh điều khiển cơ bản (F/B/L/R/S) và cập nhật cấu hình động. Khi nhận message trên topic `config`, hệ thống parse JSON và cập nhật các tham số K3, K4, K5, K6, `angle_offset` và `min_pwm` trong runtime mà không cần flash lại firmware. Mutex synchronization đảm bảo thread-safety khi cập nhật biến chia sẻ từ callback MQTT.

Firestore:

Firestore Realtime Database được tích hợp để lưu trữ dữ liệu dài hạn và cung cấp REST API cho các ứng dụng web/mobile. Khác với MQTT tập trung vào real-time messaging, Firestore phục vụ mục đích data persistence và historical analysis. Xác thực sử dụng API key với anonymous authentication, phù hợp cho thiết bị embedded không có giao diện nhập credentials.

Cấu trúc database được tổ chức theo hierarchy:

<code>/robots/bot001/info</code>	Chứa thông tin thiết bị (device ID, IP address)
<code>/robots/bot001/telemetry</code>	Lưu snapshot dữ liệu mới nhất được cập nhật mỗi 500ms
<code>/robots/bot001/config</code>	Lưu cấu hình đồng bộ với MQTT
<code>/robots/bot001/history/{session_id}/{timestamp}</code>	Lưu lịch sử theo session. Session ID được tạo từ <code>millis()</code> khi khởi động, giúp phân biệt các lần chạy khác nhau

Tần suất ghi Firestore được thiết kế hai mức: telemetry snapshot mỗi 500ms cho monitoring real-time, và history log mỗi 5 giây để tiết kiệm quota và storage. FirestoreJson object được sử dụng để serialize dữ liệu với type safety. Các trường lưu trữ bao gồm: `angle`, `pwm_l`, `pwm_r`, `enc_l`, `enc_r`, `falldown`, `balanced`, và `uptime` (thời gian hoạt động tính bằng giây).

Khi khởi động, hệ thống đọc cấu hình từ Firestore path `/robots/bot001/config` để restore các tham số K3, K4, `angle_offset` đã lưu trước đó. Điều này cho phép persistence của tuning parameters giữa các lần power cycle. Trạng thái online được set true khi kết nối thành công và tự động chuyển false khi mất kết nối nhờ Firestore SDK.

3. Kết quả và nhận xét

Hệ thống đã được triển khai và kiểm tra với các kết quả sau.

3.1. Hiệu năng cân bằng

Robot duy trì cân bằng ổn định với góc nghiêng trong khoảng $\pm 5^\circ$ khi đứng yên. Thời gian hồi phục từ nhiễu loạn nhỏ (đẩy nhẹ) khoảng 0,5-1 giây. Hệ thống tự động dừng động cơ khi góc nghiêng vượt quá 60° (phát hiện ngã).

3.2. Hiệu năng thời gian thực

Tác vụ Balance đạt jitter dưới 100 microsecond nhờ ghim vào Core 1 riêng biệt. Độ trễ từ đọc cảm biến đến xuất PWM khoảng 500 microsecond. Tổng thời gian thực thi một vòng điều khiển dưới 2ms, đảm bảo đáp ứng yêu cầu chu kỳ 3ms.

3.3. Hiệu năng truyền thông IoT

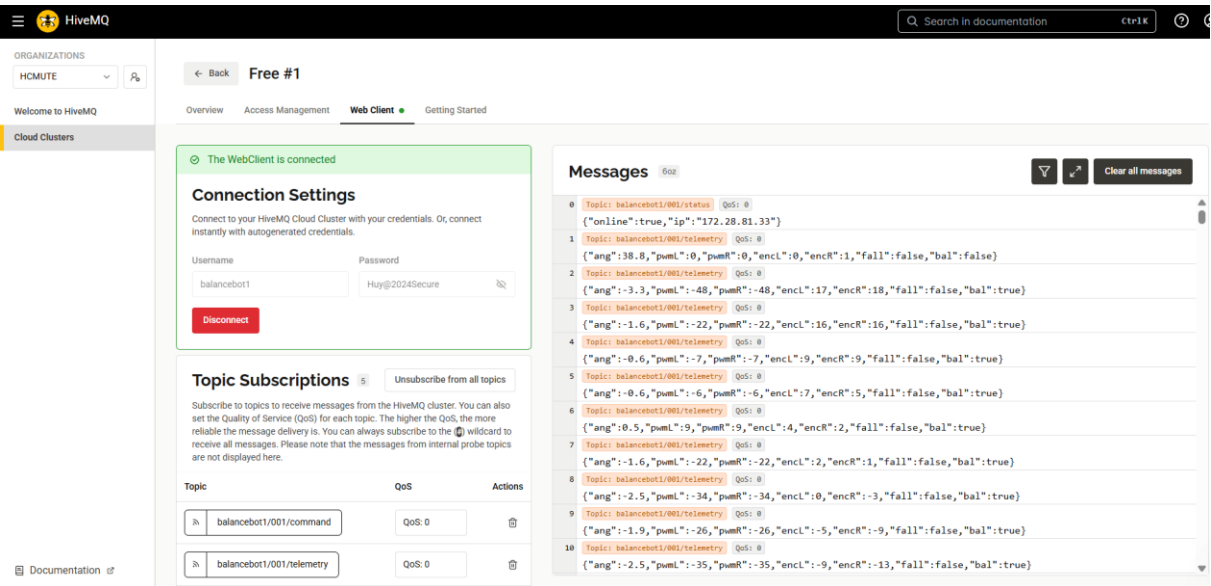
MQTT:

Kết nối MQTT qua TLS đến HiveMQ Cloud đạt độ ổn định cao với thời gian reconnect trung bình dưới 5 giây khi mất kết nối tạm thời. Handshake TLS tốn khoảng 2-3 giây do overhead mã hóa, nhưng chỉ xảy ra một lần khi thiết lập kết nối. Tần suất publish telemetry 10Hz (100ms/lần) không gây ảnh hưởng đến vòng điều khiển nhờ TaskMQTT chạy độc lập trên Core 0 với priority thấp hơn.

Độ trễ end-to-end từ publish đến subscriber nhận được dao động 50-150ms trong điều kiện mạng tốt, tăng lên 200-500ms khi mạng congestion. QoS level 0 được sử dụng cho telemetry để giảm overhead, trong khi status message sử dụng QoS 1 với retain flag. Băng thông tiêu thụ trung bình khoảng 2-3 KB/s cho telemetry stream.

Hình 1 minh họa giao diện HiveMQ Cloud Web Client với kết nối đến cluster sử dụng credentials balancebot1/Huy@2024Secure. Panel Messages hiển thị luồng message trên các topic đã subscribe: balancebot1/001/status chứa thông tin kết nối {"online":true,"ip":"172.28.81.33"} và balancebot1/001/telemetry chứa dữ liệu cảm biến dạng JSON. Có thể quan sát giá trị góc nghiêng (ang) dao động trong khoảng $-3,3^\circ$ đến $0,5^\circ$, PWM thay đổi tương ứng để duy trì cân bằng, và trạng thái bal:true cho thấy robot đang cân bằng ổn định. Topic Subscriptions cho thấy hệ thống subscribe cả command và telemetry với QoS 0.

Hình 1. Giao diện HiveMQ Cloud Web Client hiển thị luồng message MQTT



Firestore:

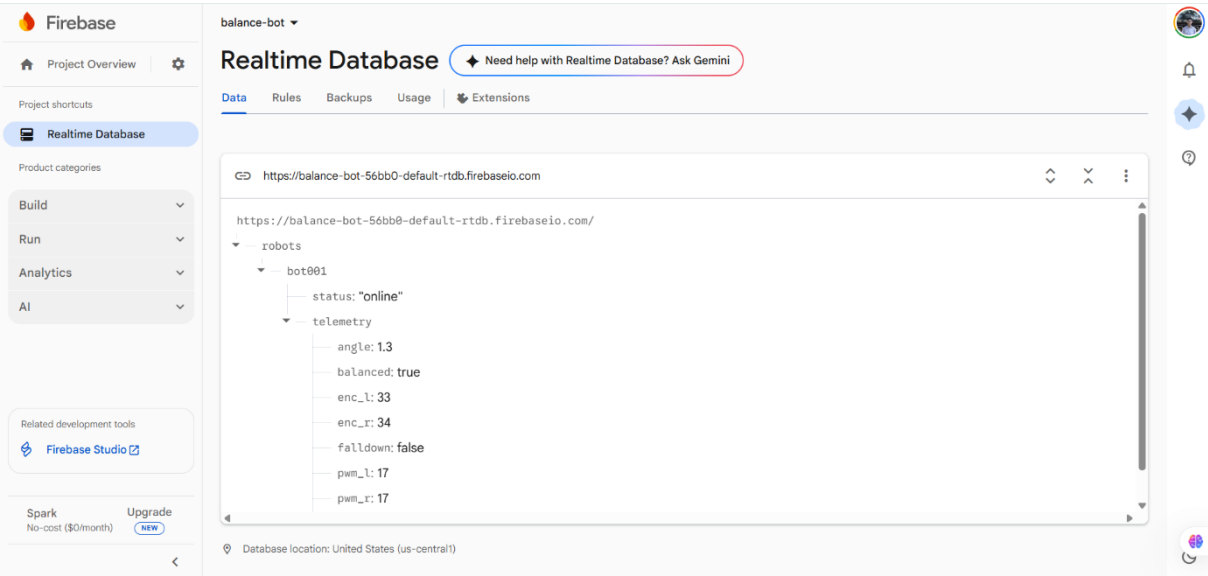
Firebase Realtime Database write latency trung bình 100-200ms cho single setJSON operation. Với tần suất ghi 2Hz (500ms/lần) cho telemetry và 0.2Hz (5s/lần) cho history, hệ thống tiêu thụ khoảng 50-100 writes/phút, nằm trong giới hạn free tier của Firebase (10GB bandwidth, 100 simultaneous connections).

Synchronization giữa MQTT config và Firebase config được thực hiện hai chiều: khi nhận config update qua MQTT, hệ thống tự động persist xuống Firebase; khi khởi động, hệ thống load config từ

Firebase. Điều này đảm bảo consistency across sessions và cho phép cấu hình từ xa qua nhiều interface khác nhau.

Hình 2 minh họa cấu trúc dữ liệu trên Firebase Realtime Database. Dữ liệu được tổ chức theo đường dẫn /robots/bot001/ với hai nhánh chính: status lưu trạng thái kết nối ("online") và telemetry lưu các thông số vận hành real-time. Các trường telemetry bao gồm: angle (góc nghiêng hiện tại), balanced (trạng thái cân bằng boolean), enc_l và enc_r (giá trị encoder trái/phải), falldown (trạng thái ngã), pwm_l và pwm_r (giá trị PWM động cơ). Dữ liệu được cập nhật liên tục với tần suất 2Hz, cho phép giám sát trạng thái robot từ xa thông qua Firebase Console hoặc ứng dụng tích hợp Firebase SDK.

Hình 2. Giao diện Firebase Realtime Database hiển thị dữ liệu telemetry thời gian thực từ robot



Bảng 3 tóm tắt các thông số kỹ thuật đạt được của hệ thống.

Bảng 3. Thông số kỹ thuật hệ thống

Thông số	Giá trị
Độ lệch góc khi cân bằng	$\pm 5^\circ$
Thời gian hồi phục	0.5 - 1.0 s
Góc phát hiện ngã	$> 60^\circ$
Jitter vòng điều khiển	$< 100 \mu s$
Tần suất MQTT telemetry	10 Hz
Độ trễ cloud command	100 - 300 ms
Chu kỳ log Firebase	5 s

4. Kết luận

Bài báo đã trình bày thiết kế và triển khai thành công hệ thống robot tự cân bằng tích hợp IoT sử dụng ESP32 với FreeRTOS. Kiến trúc đa nhiệm phân chia rõ ràng giữa điều khiển thời gian thực và truyền thông đám mây, đảm bảo hiệu năng ổn định cho cả hai chức năng. Thuật toán LQR kết hợp bộ lọc Kalman cung cấp điều khiển cân bằng tin cậy. Tích hợp MQTT và Firebase mở ra khả năng giám sát và điều khiển từ xa.

Hướng phát triển tiếp theo bao gồm: triển khai điều khiển LQR đầy đủ với 6 biến trạng thái, thêm điều khiển vị trí và quay, tích hợp camera và xử lý ảnh để tránh chướng ngại vật, và phát triển ứng dụng di động để điều khiển trực quan.

TÀI LIỆU THAM KHẢO

- [1] F. Grasser, A. D'Arrigo, S. Colombi, and A. C. Rufer, "JOE: A mobile, inverted pendulum," IEEE Trans. Ind. Electron., vol. 49, no. 1, pp. 107-114, Feb. 2002.
- [2] A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," IEEE Commun. Surveys Tuts., vol. 17, no. 4, pp. 2347-2376, 2015.
- [3] Espressif Systems, "ESP32 Technical Reference Manual," Version 4.8, 2023. [Online]. Available: <https://www.espressif.com/documentation>
- [4] R. Barry, "Mastering the FreeRTOS Real Time Kernel - A Hands On Tutorial Guide," Real Time Engineers Ltd., 2016.
- [5] G. Welch and G. Bishop, "An Introduction to the Kalman Filter," University of North Carolina at Chapel Hill, TR 95-041, 2006.
- [6] HiveMQ, "MQTT Essentials - A Lightweight IoT Messaging Protocol," 2023. [Online]. Available: <https://www.hivemq.com/mqtt-essentials/>
- [7] Firebase, "Firebase Realtime Database Documentation," Google, 2023. [Online]. Available: <https://firebase.google.com/docs/database>

TÓM TẮT TIỂU SỬ CỦA CÁC TÁC GIẢ



Le Nhat Huy Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam.

Research interests: embedded systems, IoT, robotics, design systems, semiconductors.

Email: 23119064@student.hcmute.edu.vn



Tran Hao Khiem Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, IoT, AI.



Ngô Gia Bao Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, IoT, AI.



Nguyen Le Hai Long Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, semiconductor.



Nguyen The Bao Currently pursuing Bachelor degree in Computer Engineering at Ho Chi Minh City University of Technology and Education, Vietnam

Research interests: embedded systems, IoT, AI.

LINK VIDEO MÔ PHỎNG CHI TIẾT

<https://youtu.be/nUI6qD8WDSs?si=AGAgTrQpegaPWc21>